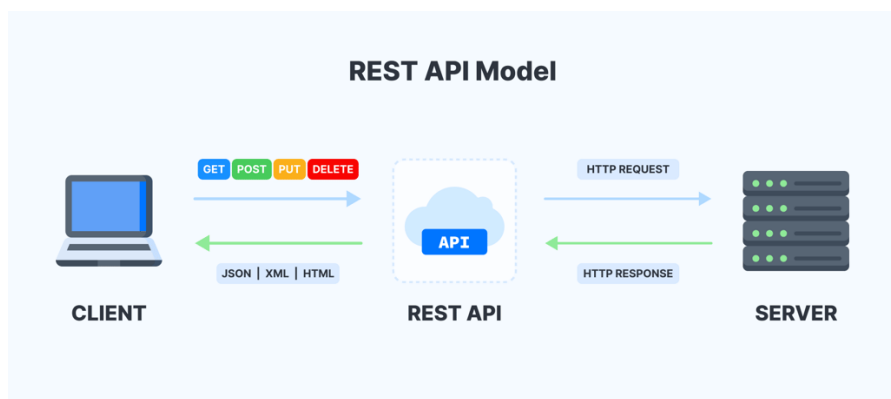




The Complete FastAPI Cheat Sheet for AI Engineers

From first endpoint to production.



Save this. Build faster.



AI Coach John



PROITBRIDGE
Shine For Better Future

Follow



Repost

What is FastAPI?

Modern Python framework for building APIs: fast, typed, async-ready.

Built on: **Starlette** + **Pydantic**

```
Flask    -> no validation, no async
Django   -> too heavy for APIs
FastAPI  -> lightweight + fast
```

Key features:

- Auto data validation
- Swagger docs at `/docs` for free
- Native async support
- Type hints = fewer bugs



Why AI Engineers Use FastAPI

- > Serve ML models as REST endpoints
- > Wrap LLM calls behind an API
- > Stream model responses
- > Build RAG pipeline endpoints
- > Create embedding APIs

Request lifecycle:

```
User sends text
      |
Validate input
      |
Run model
      |
Return response
```



AI Coach John



PROTIBRIDGE
Strive For Better Future

Follow



Repost

Installation and First App

```
pip install fastapi "uvicorn[standard]"
```

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"status": "running"}
```

```
uvicorn main:app --reload
```

- API -> **localhost:8000**
- Docs -> **localhost:8000/docs**



Project Structure

```
project/
|-- main.py
|-- routes/
|   |-- predict.py
|   |-- embeddings.py
|-- services/
|   |-- ml_service.py
|-- models/
|   |-- schemas.py
|-- core/
|   |-- config.py
```

- Routes -> HTTP logic only
- Services -> ML and business logic
- Schemas -> data validation only



Path and Query Parameters

Path parameter:

```
@app.get("/models/{name}")
def get_model(name: str):
    return {"model": name}
# GET /models/bert
```

Query parameter:

```
@app.get("/predictions")
def list_predictions(limit: int = 10):
    return {"limit": limit}
# GET /predictions?limit=5
```

FastAPI reads, types, and validates both automatically.



HTTP Methods

```
@app.get("/users/{id}")      # fetch
@app.post("/predict")        # run inference
@app.put("/users/{id}")      # update
@app.delete("/users/{id}")    # remove
```

For AI APIs:

```
GET    -> fetch results or model info
POST   -> run inference or embeddings
PUT    -> update settings
DELETE -> remove stored data
```



Pydantic Request Validation

```
from pydantic import BaseModel, Field

class PredictRequest(BaseModel):
    text: str
    temperature: float = Field(default=0.7, le=2.0)
```

```
@app.post("/predict")
def predict(request: PredictRequest):
    return model.predict(request.text)
```

Bad input is auto-rejected:

```
{"temperature": 9.0}
-> 422 Error: value must be <= 2.0
```

Your function never runs on invalid data.



Pydantic Response Schemas

```
class PredictResponse(BaseModel):  
    prediction: str  
    confidence: float
```

```
@app.post("/predict", response_model=PredictResponse)  
def predict(request: PredictRequest):  
    result = model.predict(request.text)  
    return PredictResponse(  
        prediction=result.label,  
        confidence=result.score  
    )
```

Benefits:

- Auto-filters sensitive fields
- Documents response in Swagger
- Consistent payload every time



Async Endpoints

AI calls are slow. Async handles other requests while waiting.

```
Sync:  A(3s) -> B(3s) -> C(3s) = 9s
Async: A+B+C together           = 3s
```

```
# Blocks the server
def predict(request: PredictRequest):
    return model.predict(request.text)

# Server stays free
async def predict(request: PredictRequest):
    return await model.predict(request.text)
```



Async CPU-Bound Models

ML inference blocks even async functions.

Use a thread pool to offload CPU work.

```
from concurrent.futures import ThreadPoolExecutor
import asyncio

executor = ThreadPoolExecutor(max_workers=4)

@app.post("/embed")
async def embed(request: EmbedRequest):
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(
        executor, model.encode, request.text
    )
    return {"embedding": result.tolist()}
```

```
API call -> async/await
ML model -> run_in_executor
```



Streaming Basics

Stream tokens as they generate. Don't make users wait.

```
from fastapi.responses import StreamingResponse

async def generate(prompt: str):
    async for token in llm.stream(prompt):
        yield f"data: {token}\n\n"

@app.post("/stream")
async def stream(request: ChatRequest):
    return StreamingResponse(
        generate(request.prompt),
        media_type="text/event-stream"
    )
```

Without -> user waits, text appears at once
With -> text streams word by word



Streaming With OpenAI

```
from openai import AsyncOpenAI

client = AsyncOpenAI()

async def openai_stream(messages: list):
    stream = await client.chat.completions.create(
        model="gpt-4", messages=messages, stream=True
    )
    async for chunk in stream:
        delta = chunk.choices[0].delta.content
        if delta:
            yield f"data: {delta}\n\n"
    yield "data: [DONE]\n\n"

@app.post("/chat/stream")
async def chat_stream(request: ChatRequest):
    return StreamingResponse(
        openai_stream(request.messages),
        media_type="text/event-stream"
    )
```



Dependency Injection Basics

Share logic across endpoints without repeating code.

```
def verify_key(api_key: str = Header(...)):
    if api_key not in VALID_KEYS:
        raise HTTPException(403)
    return api_key

@app.post("/predict")
def predict(
    request: PredictRequest,
    key = Depends(verify_key)
):
    return model.predict(request.text)
```

FastAPI runs **verify_key** before your function.

If it fails -> request rejected automatically.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow



Repost

Dependency Injection Model Loading

Load model once. Reuse on every request.

```
from functools import lru_cache
from sentence_transformers import SentenceTransformer

@lru_cache()
def get_model():
    return SentenceTransformer("all-MiniLM-L6-v2")

@app.post("/embed")
def embed(request: EmbedRequest, model =
Depends(get_model)):
    return {"embedding":
model.encode(request.text).tolist()}
```

Without -> reload model per request -> 10s delay
With -> load once, reuse always -> instant



Authentication **API Key**

Simple and effective for most AI APIs.

```
from fastapi.security import APIKeyHeader

api_key_header = APIKeyHeader(name="X-API-Key")

def verify_api_key(api_key = Security(api_key_header)):
    if api_key not in VALID_KEYS:
        raise HTTPException(403, "Invalid key")
    return api_key

@app.post("/predict")
def predict(request: PredictRequest, key =
Depends(verify_api_key)):
    return model.predict(request.text)
```

```
Header: X-API-Key: abc123
Valid?    -> allow
Invalid?  -> 403
```



Authentication JWT

For user-level auth : login, sessions, permissions.

```
from jose import jwt
from datetime import datetime, timedelta

def create_token(user_id: str):
    return jwt.encode(
        {"sub": user_id, "exp": datetime.utcnow() +
        timedelta(hours=24)},
        SECRET, algorithm="HS256"
    )

def verify_token(token = Depends(oauth2_scheme)):
    try:
        payload = jwt.decode(token, SECRET, ["HS256"])
        return payload.get("sub")
    except:
        raise HTTPException(401)
```

Login -> token issued
Every request -> token verified
Expired/invalid -> 401



Rate Limiting Setup

Protect your API from abuse and runaway costs.

```
pip install slowapi
```

```
from slowapi import Limiter,  
_rate_limit_exceeded_handler  
from slowapi.util import get_remote_address  
  
limiter = Limiter(key_func=get_remote_address)  
app.state.limiter = limiter  
app.add_exception_handler(RateLimitExceeded,  
_rate_limit_exceeded_handler)  
  
@app.post("/predict")  
@limiter.limit("10/minute")  
async def predict(request: Request, data:  
PredictRequest):  
    return model.predict(data.text)
```

Over limit -> **429 Too Many Requests**



Rate Limiting Tiered Plans

```
def get_limit(request: Request):  
    plan = get_plan(request.headers.get("X-API-Key"))  
    return {  
        "free": "10/minute",  
        "pro": "100/minute"  
    }.get(plan, "5/minute")  
  
@app.post("/generate")  
@limiter.limit(get_limit)  
async def generate(request: Request, data:  
    GenerateRequest):  
    return await generate_text(data.prompt)
```

```
Free key    -> 10/min  
Pro key     -> 100/min  
No key      -> 5/min  
Over limit  -> 429
```



Middleware **CORS** and **Timing**

Runs on every request before reaching your endpoint.

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=["https://yourapp.com"],  
    allow_methods=["*"],  
    allow_headers=["*"]  
)
```

```
@app.middleware("http")  
async def add_timing(request: Request, call_next):  
    start = time.time()  
    response = await call_next(request)  
    ms = (time.time() - start) * 1000  
    response.headers["X-Response-Time"] = f"{ms:.0f}ms"  
    return response
```



Background Tasks

Run tasks after sending response. User doesn't wait.

```
from fastapi import BackgroundTasks

def save_log(text: str, result: str):
    db.save(text=text, result=result)

@app.post("/predict")
async def predict(
    request: PredictRequest,
    background_tasks: BackgroundTasks
):
    result = model.predict(request.text)
    background_tasks.add_task(save_log, request.text,
    result)
    return {"prediction": result}
```

Response sent immediately
Logging runs after in background



File Uploads

Accept PDFs and images for AI document pipelines.

```
from fastapi import UploadFile, File
import pypdf, io

@app.post("/process-pdf")
async def process_pdf(file: UploadFile = File(...)):
    if file.content_type != "application/pdf":
        raise HTTPException(400, "PDFs only")

    contents = await file.read()
    pdf = pypdf.PdfReader(io.BytesIO(contents))
    text = "".join(p.extract_text() for p in pdf.pages)

    return {"pages": len(pdf.pages), "chars": len(text)}
```

Always validate **content_type** before processing.



AI Coach John



PROITBRIDGE
Shine For Better Future

Follow

 Repost

Caching In Memory

Never run the same computation twice.

```
import hashlib

cache = {}

@app.post("/embed")
async def embed(request: EmbedRequest):
    key = hashlib.md5(request.text.encode()).hexdigest()

    if key in cache:
        return {"embedding": cache[key], "cached": True}

    result = model.encode(request.text).tolist()
    cache[key] = result
    return {"embedding": result, "cached": False}
```

```
Same text -> cache hit -> instant
New text -> run model -> store result
```



Caching Redis

For production and multi-server deployments.

```
import redis, json

r = redis.Redis(host="localhost", port=6379)

@app.post("/embed")
async def embed(request: EmbedRequest):
    key = f"embed:
{hashlib.md5(request.text.encode()).hexdigest()}"
    cached = r.get(key)

    if cached:
        return {"embedding": json.loads(cached),
"cached": True}

    result = model.encode(request.text).tolist()
    r.setex(key, 3600, json.dumps(result)) # 1hr TTL
    return {"embedding": result, "cached": False}
```

In-memory -> single server only

Redis -> shared across all servers



Error Handling

Never expose raw errors. Return clean structured responses.

```
@app.post("/predict")
async def predict(request: PredictRequest):
    if len(request.text) > 10000:
        raise HTTPException(400, "Text too long")

    if request.model not in MODELS:
        raise HTTPException(404, "Model not found")

    return model.predict(request.text)

@app.exception_handler(Exception)
async def global_handler(request: Request, exc:
Exception):
    logger.error(f"Error: {exc}")
    return JSONResponse(500, {"detail": "Something went
wrong"})
```

Common status codes:

```
400 -> bad input    404 -> not found
429 -> rate limit    503 -> model loading
```



Serving ML Models

Load model once at startup. Never per request.

```
from contextlib import asynccontextmanager

models = {}

@asynccontextmanager
async def lifespan(app: FastAPI):
    models["embedder"] = SentenceTransformer("all-
MiniLM-L6-v2")
    yield
    models.clear()

app = FastAPI(lifespan=lifespan)

@app.post("/embed")
async def embed(request: EmbedRequest):
    return {"embedding":
models["embedder"].encode(request.text).tolist()}
```

Startup -> load once
Requests -> reuse from memory
Shutdown -> cleanup



Health Checks

Load balancers and Kubernetes rely on these.

```
@app.get("/health")
async def health():
    return {"status": "alive"}

@app.get("/health/ready")
async def ready():
    if not models:
        raise HTTPException(503, "Models not loaded")
    return {"status": "ready", "models":
list(models.keys())}
```

```
/health      -> is app alive?
/health/ready -> is app ready for traffic?
```

```
200 -> send traffic
503 -> remove from rotation
```



Environment Config

Never hardcode secrets. Use env variables.

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    openai_api_key: str
    redis_url: str = "redis://localhost:6379"
    default_model: str = "gpt-4"
    debug: bool = False

    class Config:
        env_file = ".env"

@lru_cache()
def get_settings():
    return Settings()
```

.env file:

```
OPENAI_API_KEY=sk-xxxxxxx
DEBUG=false
```

Inject with **Depends(get_settings)** anywhere.



AI Coach John



Follow

 Repost

Routers Organizing Large APIs

Split endpoints into feature modules.

```
# routes/embeddings.py
router = APIRouter(prefix="/embeddings", tags=
["Embeddings"])

@router.post("/")
async def create_embedding(request: EmbedRequest):
    return {"embedding":
model.encode(request.text).tolist()}
```

```
# main.py
from routes import embeddings, chat, documents

app.include_router(embeddings.router, prefix="/api/v1")
app.include_router(chat.router, prefix="/api/v1")
```

One file -> 500 lines mixed
Routers -> clean modules



Testing

```
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_health():
    res = client.get("/health")
    assert res.status_code == 200

def test_predict():
    res = client.post(
        "/predict",
        json={"text": "hello"},
        headers={"X-API-Key": "test-key"}
    )
    assert res.status_code == 200
    assert "prediction" in res.json()

def test_missing_auth():
    res = client.post("/predict", json={"text":
"hello"})
    assert res.status_code == 403
```



Deployment Docker

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0",
    "--port", "8000", "--workers", "4"]
```

```
# docker-compose.yml
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [redis]
  redis:
    image: redis:7-alpine
```

Workers rule:

```
I/O-bound -> CPU cores * 2 + 1
CPU-bound -> CPU cores + 1
```



AI Coach John



Follow

 Repost